

Fall 2024

CE Department
Sharif University of Technology

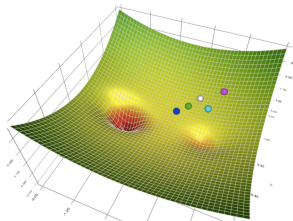
October 22, 2024



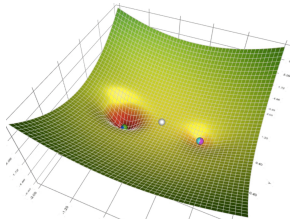
4 References

How to Update Weights?

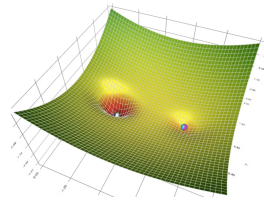
- Imagine training a large model like ChatGPT. It has billions of parameters that need to be adjusted.
- If we used **random search** to update these weights, it would take an astronomical number of trials to find good parameters.
- How to make training feasible at this massive scale?
- Image adapted from Medium: AdaGrad Optimization Algorithm



Visualizing parameter space with
random initial points.



Random search approach in
high-dimensional space.



Challenges in finding optimal weights using random search.

How to Update Weights?

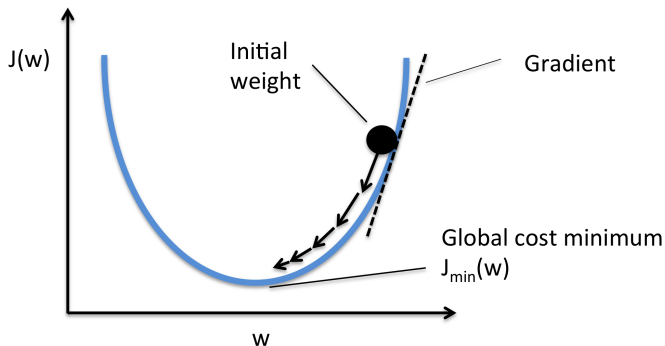
Options for Updating Weights:

- **Random Search:** Tries values randomly—inefficient and impractical.
- **Gradient Descent:** Follows the slope of the loss function—efficient and guided.

Why Gradient Descent?

- It updates weights by following the slope, reducing error with each step.
- Controlled, stepwise updates ensure we move closer to minimizing the loss effectively.

How to Update Weights?



Gradient descent: adjusting weights to minimize cost by following the gradient.

Gradient optimization image

Gradient Descent: Concept and Weight Updates

Gradient Descent: Minimizes the loss function by updating weights based on the gradient.

Weight Update Rule:

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \frac{\partial L}{\partial w}$$

Where:

- η is the learning rate (step size).
- $\frac{\partial L}{\partial w}$ is the gradient of the loss function with respect to w .

2 Backpropagation

Forward and Backward Passes

Vectorized Backpropagation

4 References

① Gradient Descent

② Backpropagation

Forward and Backward Passes

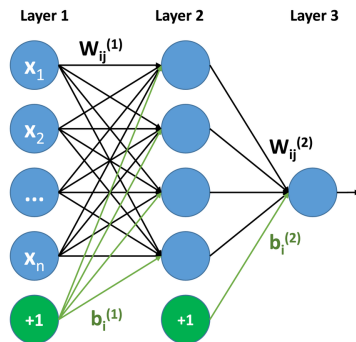
Vectorized Backpropagation

③ Foundations in Detail: Initialization, Loss, and Activation

④ References

Multi-Layer Neural Network

The diagram below illustrates a three-layer neural network

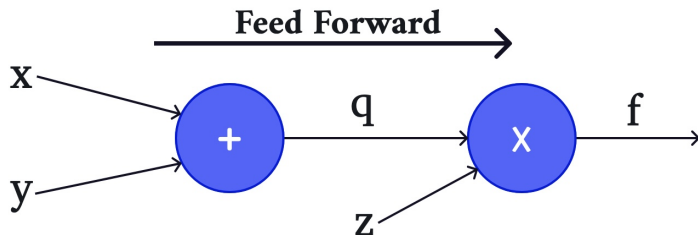


Neural Network

How should $\frac{\partial L}{\partial w}$ be calculated for each weight w in this neural network?

$$f(x, y, z) = (x + y)z$$

$$f(x, y, z) = (x + y)z$$



$$f(x, y, z) = (x + y)z$$

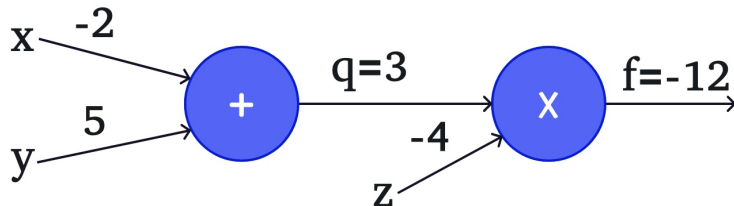
$$f(x, y, z) = (x + y)z$$

$$x = -2, \quad y = 5, \quad z = -4$$

$$x = -2, \quad y = 5, \quad z = -4$$

- $\mathbf{q} = \mathbf{x} + \mathbf{y} = 3$
- $\mathbf{f} = \mathbf{q} \times \mathbf{z} = -12$

- $\mathbf{q} = \mathbf{x} + \mathbf{y} = 3$
- $\mathbf{f} = \mathbf{q} \times \mathbf{z} = -12$



Backpropagation

A more efficient method:

- Use the chain rule to compute all gradients in one backward pass.
- This method avoids changing each weight separately.

Advantages:

- Complexity is reduced to $\Theta(n)$.
- Much faster, especially for large neural networks.

Chain Rule for Gradients

The **chain rule** helps us find the gradient of a function that is composed of other functions.

Example:

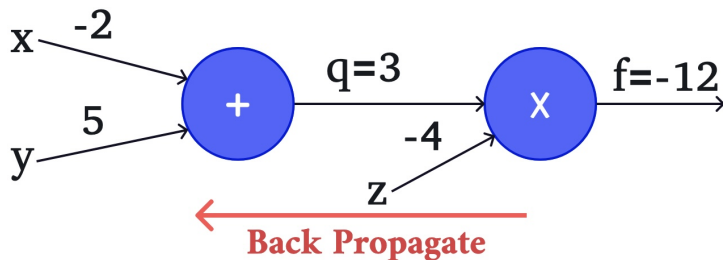
$$z = f(g(x))$$

- f is a function of $g(x)$
- $g(x)$ is a function of x

To find $\frac{\partial z}{\partial x}$, we use the chain rule:

$$\frac{\partial z}{\partial x} = \frac{\partial f}{\partial g} \times \frac{\partial g}{\partial x}$$

This means we multiply the gradient of the outer function by the gradient of the inner function.

$$f(x, y, z) = (x + y)z$$
$$x = -2, \quad y = 5, \quad z = -4$$


Function:

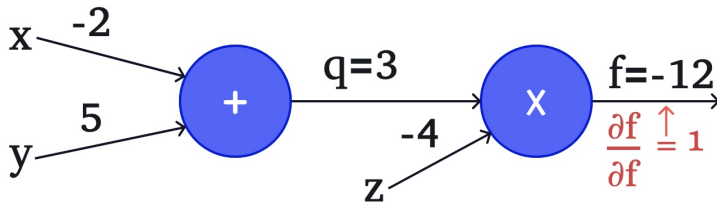
$$f(x, y, z) = (x + y)z$$

Example:

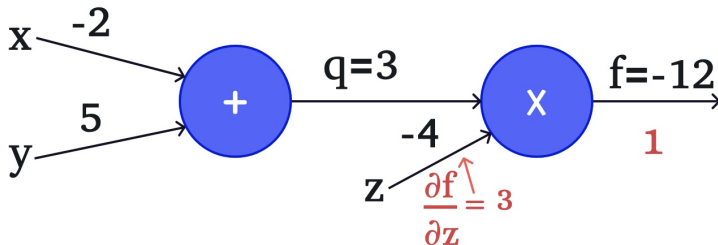
$$x = -2, \quad y = 5, \quad z = -4$$

Step 1:

$$\frac{\partial f}{\partial f} = 1$$



1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 1000 1001 1002 1003 1004 1005 1006 1007 1008 1009 1010 1011 1012 1013 1014 1015 1016 1017 1018 1019 1020 1021 1022 1023 1024 1025 1026 1027 1028 1029 1030 1031 1032 1033 1034 1035 1036 1037 1038 1039 1040 1

$$f(x, y, z) = (x + y)z$$
$$x = -2, \quad y = 5, \quad z = -4$$
$$f = qz, \quad \frac{\partial f}{\partial z} = q = 3$$


$$f(x, y, z) = (x + y)z$$

$$f(x, y, z) = (x + y)z$$

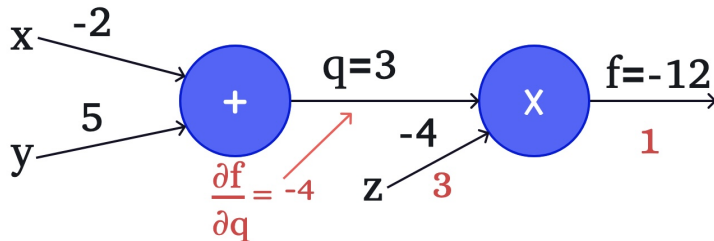
$$x = -2, \quad y = 5, \quad z = -4$$

$$x = -2, \quad y = 5, \quad z = -4$$

$$f = qz, \quad \frac{\partial f}{\partial z} = q = 3$$

$$f = qz, \quad \frac{\partial f}{\partial z} = q = 3$$

$$\frac{\partial f}{\partial q} = z = -4$$



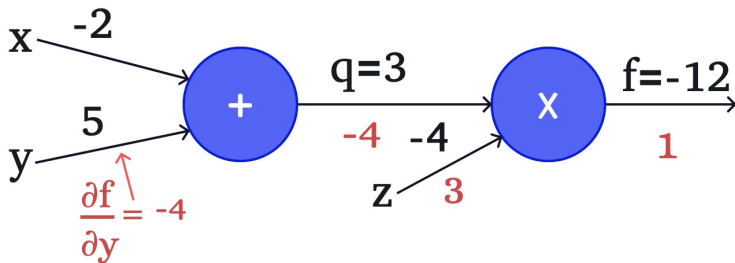
$$f(x, y, z) = (x + y)z$$

$$f(x, y, z) = (x + y)z$$

$$x = -2, \quad y = 5, \quad z = -4$$

$$q = x + y, \quad \frac{\partial q}{\partial x} = 1, \quad \frac{\partial q}{\partial y} = 1$$

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y} = -4 \cdot 1 = -4$$



$$f(x, y, z) = (x + y)z$$

$$f(x, y, z) = (x + y)z$$

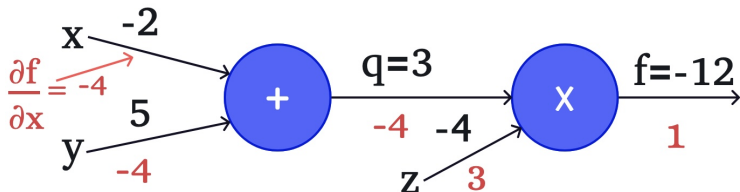
$$x = -2, \quad y = 5, \quad z = -4$$

$$x = -2, \quad y = 5, \quad z = -4$$

$$q = x + y, \quad \frac{\partial q}{\partial x} = 1, \quad \frac{\partial q}{\partial y} = 1$$

$$q = x + y, \quad \frac{\partial q}{\partial x} = 1, \quad \frac{\partial q}{\partial y} = 1$$

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x} = -4 \cdot 1 = -4$$



$$f(x, y, z) = (x + y)z$$

$$f(x, y, z) = (x + y)z$$

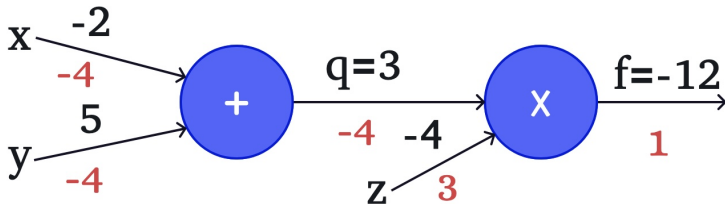
$$x = -2, \quad y = 5, \quad z = -4$$

$$x = -2, \quad y = 5, \quad z = -4$$

$$q = x + y, \quad \frac{\partial q}{\partial x} = 1, \quad \frac{\partial q}{\partial y} = 1$$

$$q = x + y, \quad \frac{\partial q}{\partial x} = 1, \quad \frac{\partial q}{\partial y} = 1$$

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x} = -4 \cdot 1 = -4$$



Example: One Neuron

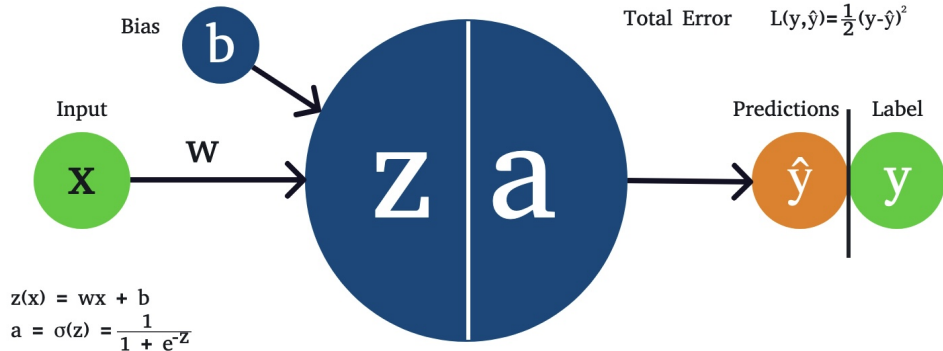
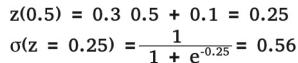
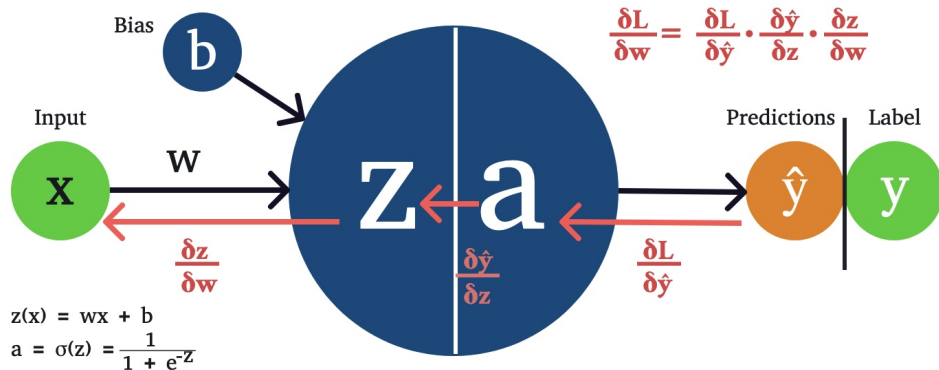


Image adapted from “Deep learning backpropagation” Web article



Backward Pass



① Gradient Descent

② Backpropagation

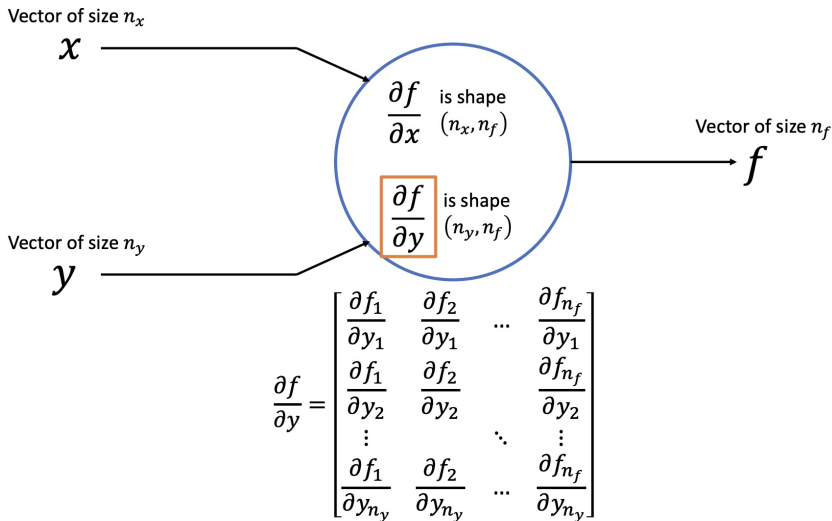
Forward and Backward Passes

Vectorized Backpropagation

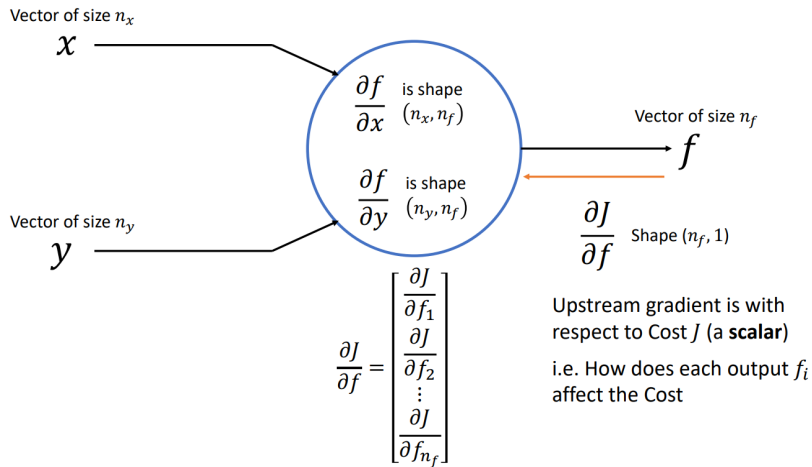
③ Foundations in Detail: Initialization, Loss, and Activation

④ References

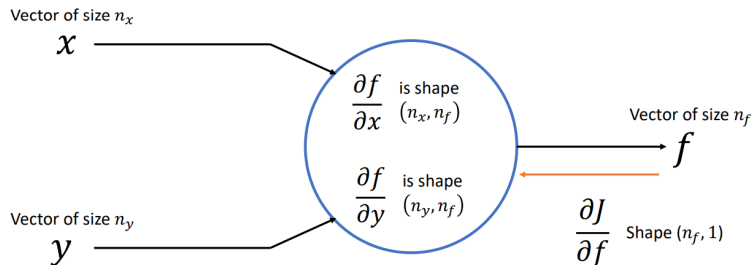
Vectorized Backpropagation



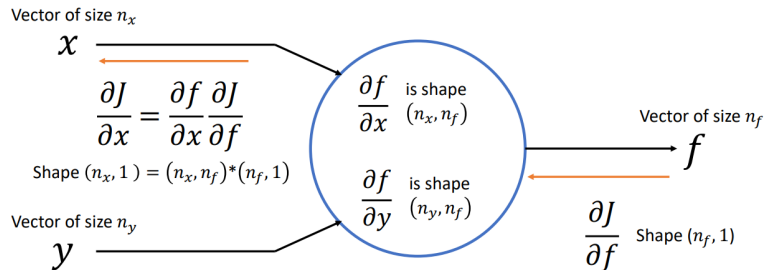
Vectorized Backpropagation



Vectorized Backpropagation



Apply chain rule like before!



Applying the chain rule involves matrix-vector multiplication

Ali Sharifi-Zarchi (Sharif University of Technology)



◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ◻ ◻ ◻ ↺ 🔍 ↻

Chain Rule – Matrix-Vector Multiply

$$\frac{\partial J}{\partial x} = \frac{\partial f}{\partial x} \frac{\partial J}{\partial f} \rightarrow \begin{bmatrix} \frac{\partial J}{\partial x_1} \\ \frac{\partial J}{\partial x_2} \\ \vdots \\ \frac{\partial J}{\partial x_{n_x}} \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1}, \frac{\partial f_2}{\partial x_1}, & \dots & \frac{\partial f_{n_f}}{\partial x_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_1}{\partial x_{n_x}}, \frac{\partial f_2}{\partial x_{n_x}} & \dots & \frac{\partial f_{n_f}}{\partial x_{n_x}} \end{bmatrix} \begin{bmatrix} \frac{\partial J}{\partial f_1} \\ \frac{\partial J}{\partial f_2} \\ \vdots \\ \frac{\partial J}{\partial f_{n_f}} \end{bmatrix}$$

$$\text{Shape } (n_x, 1) = (n_x, n_f) * (n_f, 1)$$

Chain Rule – Matrix-Vector Multiply

$$\frac{\partial J}{\partial x} = \frac{\partial f}{\partial x} \frac{\partial J}{\partial f} \rightarrow \begin{bmatrix} \frac{\partial J}{\partial x_1} \\ \frac{\partial J}{\partial x_2} \\ \vdots \\ \frac{\partial J}{\partial x_{n_x}} \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1}, \frac{\partial f_2}{\partial x_1}, & \dots & \frac{\partial f_{n_f}}{\partial x_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_1}{\partial x_{n_x}}, \frac{\partial f_2}{\partial x_{n_x}} & \dots & \frac{\partial f_{n_f}}{\partial x_{n_x}} \end{bmatrix} \begin{bmatrix} \frac{\partial J}{\partial f_1} \\ \frac{\partial J}{\partial f_2} \\ \vdots \\ \frac{\partial J}{\partial f_{n_f}} \end{bmatrix}$$

Jacobian

Shape $(n_x, 1) = (n_x, n_f) * (n_f, 1)$

Chain Rule – Matrix-Vector Multiply

The diagram illustrates the chain rule for backpropagation through a Jacobian matrix. It shows the relationship between the total derivative of the loss J with respect to an input x and the upstream gradient.

The equation shown is:

$$\frac{\partial J}{\partial x} = \frac{\partial f}{\partial x} \frac{\partial J}{\partial f} \rightarrow \begin{bmatrix} \frac{\partial J}{\partial x_1} \\ \frac{\partial J}{\partial x_2} \\ \vdots \\ \frac{\partial J}{\partial x_{n_x}} \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1}, \frac{\partial f_2}{\partial x_1}, & \dots & \frac{\partial f_{n_f}}{\partial x_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_1}{\partial x_{n_x}}, \frac{\partial f_2}{\partial x_{n_x}} & \dots & \frac{\partial f_{n_f}}{\partial x_{n_x}} \end{bmatrix} \begin{bmatrix} \frac{\partial J}{\partial f_1} \\ \frac{\partial J}{\partial f_2} \\ \vdots \\ \frac{\partial J}{\partial f_{n_f}} \end{bmatrix}$$

Annotations in the diagram include:

- Jacobian**: A blue arrow points from the Jacobian matrix to the $\frac{\partial f}{\partial x}$ term in the equation.
- Upstream Gradient**: An orange arrow points from the $\frac{\partial J}{\partial f}$ term in the equation to the $\frac{\partial J}{\partial f}$ term in the final product.
- Shape**: The text $\text{Shape } (n_x, 1) = (n_x, n_f) * (n_f, 1)$ is shown below the Jacobian matrix, indicating the dimensions of the vectors and the matrix multiplication.

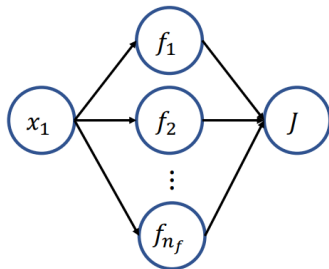
Chain Rule – Matrix-Vector Multiply

$$\frac{\partial J}{\partial x} = \frac{\partial f}{\partial x} \frac{\partial J}{\partial f} \rightarrow \begin{bmatrix} \frac{\partial J}{\partial x_1} \\ \frac{\partial J}{\partial x_2} \\ \vdots \\ \frac{\partial J}{\partial x_{n_x}} \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1}, \frac{\partial f_2}{\partial x_1}, & \dots & \frac{\partial f_{n_f}}{\partial x_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_1}{\partial x_{n_x}}, \frac{\partial f_2}{\partial x_{n_x}} & \dots & \frac{\partial f_{n_f}}{\partial x_{n_x}} \end{bmatrix} \begin{bmatrix} \frac{\partial J}{\partial f_1} \\ \frac{\partial J}{\partial f_2} \\ \vdots \\ \frac{\partial J}{\partial f_{n_f}} \end{bmatrix}$$

$$\text{Shape } (n_x, 1) = (n_x, n_f) * (n_f, 1)$$

$$\frac{\partial J}{\partial x_1} = \frac{\partial f_1}{\partial x_1} \frac{\partial J}{\partial f_1} + \frac{\partial f_2}{\partial x_1} \frac{\partial J}{\partial f_2} + \dots + \frac{\partial f_{n_f}}{\partial x_1} \frac{\partial J}{\partial f_{n_f}}$$

Chain Rule – Matrix-Vector Multiply

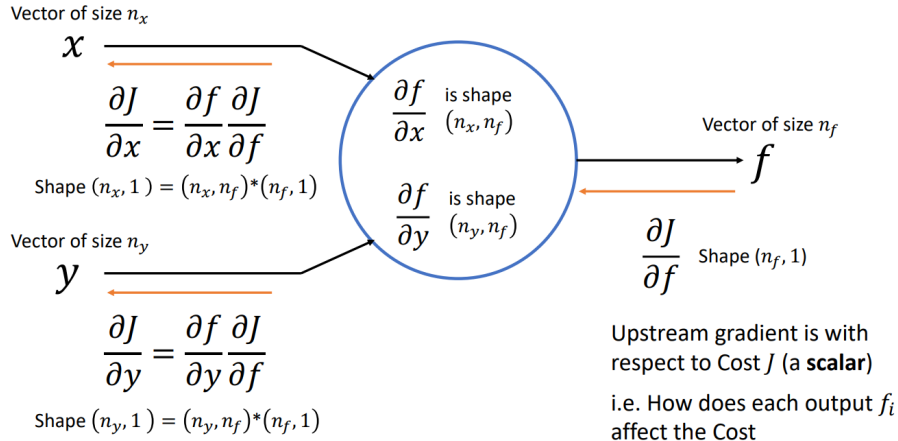


$$\frac{\partial J}{\partial x} = \frac{\partial f}{\partial x} \frac{\partial J}{\partial f} \rightarrow$$

$$\begin{bmatrix} \frac{\partial J}{\partial x_1} \\ \frac{\partial J}{\partial x_2} \\ \vdots \\ \frac{\partial J}{\partial x_{n_x}} \end{bmatrix} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1}, \frac{\partial f_2}{\partial x_1}, & \dots & \frac{\partial f_{n_f}}{\partial x_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_1}{\partial x_{n_x}}, \frac{\partial f_2}{\partial x_{n_x}} & \dots & \frac{\partial f_{n_f}}{\partial x_{n_x}} \end{bmatrix} \begin{bmatrix} \frac{\partial J}{\partial f_1} \\ \frac{\partial J}{\partial f_2} \\ \vdots \\ \frac{\partial J}{\partial f_{n_f}} \end{bmatrix}$$

$$\text{Shape } (n_x, 1) = (n_x, n_f) * (n_f, 1)$$

$$\frac{\partial J}{\partial x_1} = \frac{\partial f_1}{\partial x_1} \frac{\partial J}{\partial f_1} + \frac{\partial f_2}{\partial x_1} \frac{\partial J}{\partial f_2} + \dots + \frac{\partial f_{n_f}}{\partial x_1} \frac{\partial J}{\partial f_{n_f}}$$



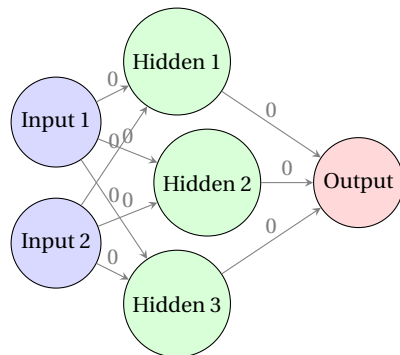
Chain Rule application is Matrix-Vector Multiply

Weight Initialization

Example: Imagine a network where all weights are initialized to zero.

Issue: If all weights are zero, each neuron in a layer will produce **identical** outputs. This symmetry prevents the network from learning **distinct features**, as every neuron updates identically.

Solution: To break this symmetry, weights need to be initialized with small random values, allowing neurons to learn unique features and avoid identical updates.



All weights initialized to zero

Why Weight Initialization Matters

Importance:

- Proper **initialization** ensures **faster convergence** and improves **training stability**.
- Prevents issues like **vanishing** or **exploding gradients**, which can make training slow or unstable.

Question: How can we initialize weights to maximize **learning efficiency** and prevent gradient problems?

Zero Initialization and Random Initialization

Zero Initialization

- **Description:** Set all weights to zero.
- **Key Point:** Rarely used, as it leads to identical updates for all neurons, preventing the network from learning distinct features.

Random Initialization

- **Description:** Assign small random values to weights.
- **Distribution:** Typically, weights are initialized using a uniform or normal distribution.

$$w \sim \mathcal{U}(-\epsilon, \epsilon) \quad \text{or} \quad w \sim \mathcal{N}(0, \sigma^2)$$

- **Key Point:** Helps break symmetry but can still cause issues with gradient magnitudes.

Xavier Initialization

Description: Xavier Initialization is designed to keep the variance of activations consistent across layers, ideal for **sigmoid** and **tanh** activations.

Objective: Prevents the shrinking or exploding of signal magnitudes during forward and backward propagation.

Condition:

$$\frac{1}{n_l} \text{Var}[w] = 1$$

where n_l is the number of neurons in layer l .

Initialization Scheme:

$$w \sim \mathcal{U}\left(-\sqrt{\frac{1}{n_l}}, \sqrt{\frac{1}{n_l}}\right)$$

This results in a uniform distribution within the range $-\sqrt{\frac{1}{n_l}}$ to $\sqrt{\frac{1}{n_l}}$, ensuring stable signal variance across layers.

He Initialization

Description: He Initialization (or Kaiming Initialization) is designed for neural networks with **ReLU** activations, considering the non-linearity of these functions.

Objective: Aims to prevent the exponential growth or reduction of input signal magnitudes through layers.

Condition:

$$\frac{1}{2} n_l \text{Var}[w] = 1$$

Initialization Scheme:

$$w_l \sim \mathcal{N}\left(0, \frac{2}{n_l}\right)$$

This implies a zero-centered Gaussian distribution with a standard deviation of $\sqrt{\frac{2}{n_l}}$, where biases are initialized to 0.

- ① Gradient Descent
- ② Backpropagation
- ③ **Foundations in Detail: Initialization, Loss, and Activation**
 - Weight Initialization
 - Loss Functions**
 - Activation Functions
- ④ References

Mean Squared Error (MSE)

Definition:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N \left(y^{(i)} - \hat{y}^{(i)} \right)^2$$

Characteristics:

- Amplifies larger errors due to squaring, making it sensitive to outliers.

Example of MSE Calculation

Example:

- Predicted values: $\hat{y} = [4.2, 3.8, 5.1]$
- True values: $y = [5.0, 4.0, 4.9]$
- Calculation:

$$\begin{aligned}\text{MSE} &= \frac{1}{3} [(5.0 - 4.2)^2 + (4.0 - 3.8)^2 + (4.9 - 5.1)^2] \\ &= \frac{1}{3} [0.64 + 0.04 + 0.04] = \frac{0.72}{3} = 0.24\end{aligned}$$

Categorical Cross-Entropy

Categorical Cross-Entropy Formula:

$$\mathcal{L}_{CCE} = -\frac{1}{n} \sum_{i=1}^n \sum_{c=1}^C y_c^{(i)} \log(\hat{y}_c^{(i)})$$

- ③ Compute log terms:

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\mathcal{L}_{CCE} = -\frac{1}{3} (\log(0.7) + \log(0.6) + \log(0.3))$$

$$\log(0.7) \approx -0.357, \quad \log(0.6) \approx -0.511, \quad \log(0.3) \approx -1.204$$

$$\mathcal{L}_{CCE} = \frac{1}{3} \times 2.072 \approx 0.691$$

Limitation of Linear Activation

Why Transform Outputs? Raw outputs need to be transformed into meaningful values, such as probabilities.

The Problem: Linear activation lacks non-linearity, restricting the model to simple linear relationships.



Neural Networks: Why is the Max Operator Important?

- **Before:** Linear score function:

$$f = Wx$$

- **Now:** 2-layer Neural Network:

$$f = W_2 \max(0, W_1 x)$$

- The function $\max(0, z)$ is called an activation function (in this case, ReLU).
- **Q: What if we try to build a neural network without an activation function?**

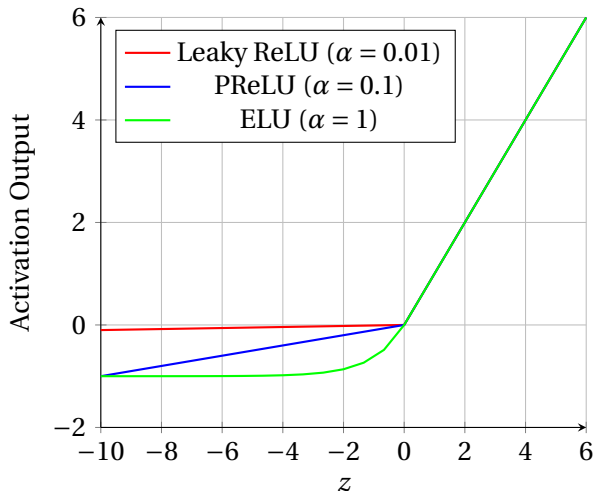
$$f = W_2 W_1 x$$

$$W_3 = W_2 W_1 \in \mathbb{R}^{C \times H}, \quad f = W_3 x$$

- **A:** We end up with a linear classifier again!

Variants of ReLU: Leaky ReLU, PReLU, ELU

Leaky ReLU, PReLU, and ELU Activation Functions

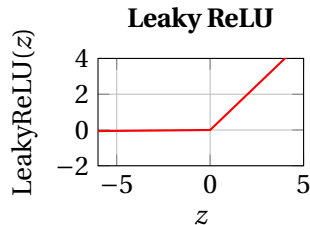


Variants of ReLU: Leaky ReLU

- Allows a small, non-zero gradient for negative inputs.

$$\text{LeakyReLU}(z) = \max(\alpha z, z), \quad \alpha = 0.01$$

- Helps prevent the "dead ReLU" problem, where neurons stop updating.



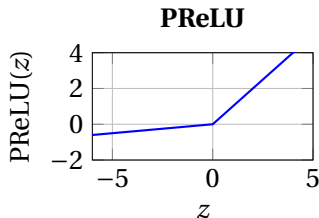
Leaky ReLU: Allows a small, non-zero gradient for negative inputs.

Variants of ReLU: PReLU (Parametric ReLU)

- Similar to Leaky ReLU, but the slope for negative inputs (α) is learned during training.

$$\text{PReLU}(z) = \max(\alpha z, z), \quad \alpha \text{ is learned}$$

- Provides more flexibility by adjusting the slope for negative inputs based on data.



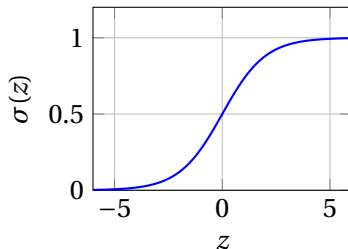
PReLU: Similar to Leaky ReLU, with a learnable slope.

Sigmoid

Characteristics of Sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

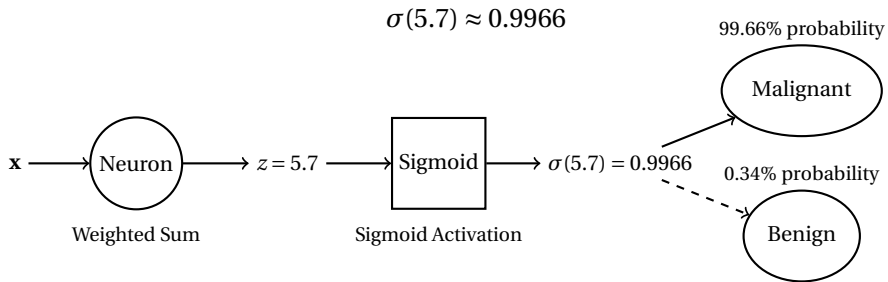
- $\sigma'(z) = \sigma(z)(1 - \sigma(z))$
- Squashes the input between 0 and 1, which makes it useful in probabilistic interpretations (e.g., logistic regression).
- Often used in output layers for binary classification problems.



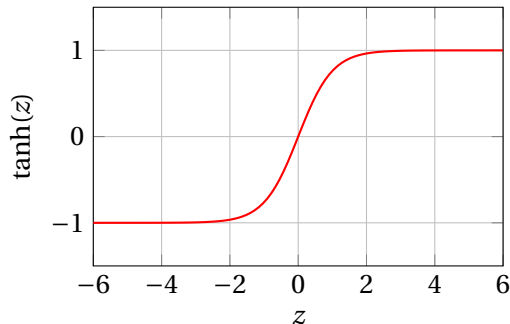
Classification: Tumor Detection (Malignant vs. Benign)

Sigmoid Activation: Useful for binary classification!

- Example: For $z = 5.7$,



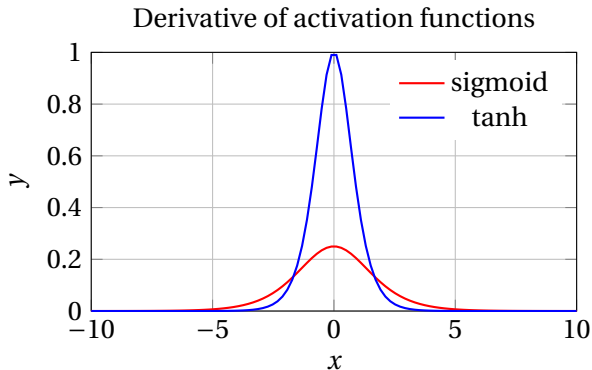
Tanh (Hyperbolic Tangent)



Question:

- How does Tanh help with faster convergence compared to Sigmoid?

Comparison: Sigmoid vs Tanh



- The derivative of the Tanh function has a much steeper slope at $x = 0$, meaning it provides a larger gradient for backpropagation compared to the Sigmoid function.

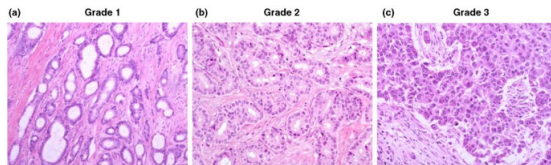
Problem: Multi-Class Tumor Classification

Scenario: We want to classify a tumor into one of three categories:

- **Class 0:** Benign
- **Class 1:** Malignant
- **Class 2:** Pre-cancerous

Goal: Given a set of tumor features, predict which class the tumor belongs to.

This is a **multi-class classification problem**, and we will use the **Softmax activation function** to assign probabilities to each class.



Microscopic images of tumor tissue, classified into grades representing different severity levels. Image adapted from Visual Analytics in Digital Computational Pathology

Example: Softmax Probabilities

Step 3: Calculate Softmax probabilities for each class

$$P(\text{Benign}) = \frac{4.48}{7.32} \approx 0.612, \quad P(\text{Malignant}) = \frac{2.23}{7.32} \approx 0.305, \quad P(\text{Pre-Cancerous}) = \frac{0.61}{7.32} \approx 0.083$$

Step 4: Make a classification decision

- The highest probability is 0.612 for the **Benign** class.
- Therefore, the model predicts that the tumor is **Benign** (Class 0).

- A set of small navigation icons typically found in Beamer presentations, including symbols for back, forward, search, and other slide controls.

- [1] T. Networks, “Benign vs malignant tumors.” *Web article*, 2023.
<https://www.technologynetworks.com/cancer-research/articles/benign-vs-malignant-tumors-364765>.
- [2] B. Central, “Breast cancer research article.” *Journal article*, 2010.
<https://breast-cancer-research.biomedcentral.com/articles/10.1186/bcr2607>.
- [3] S. Raschka, “Gradient optimization image.” *Image*, 2020.
<https://sebastianraschka.com/images/faq/gradient-optimization/ball.png>.
- [4] Xnought, “Backpropagation explainer.” *Website*, 2022.
<https://xnought.github.io/backprop-explainer/>.
- [5] Datamapu, “Deep learning backpropagation.” *Web article*, 2021.
https://datamapu.com/posts/deep_learning/backpropagation/.

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

Any Questions?